INDUSTRIAL TRAINING PRESENTATION (ITR) · 2026

SpendMind

Financial Data Pipeline & NLP-Based Transaction Categorization Engine.

SUBMITTED BY

Somya Roy

Roll No.: **[Your Roll No.]** · github.com/Missroy1905

SUBMITTED TO

[Faculty Name]

Department of Computer Science & Engineering

TECHNICAL STACK: PYTHON 3.11 · PDFPLUMBER · SCIKIT-LEARN · FASTAPI · SQLITE

STRUCTURE & ROADMAP

Table of Contents

A systematic overview of the training tenure, engineering architecture, and software deliverables.

01
Company Profile
Corporate introduction, operational footprint, domains, and client industries.

02
Certificate
Official internship certification and organizational evaluation proof.

03
Technology Stack
Python, pdfplumber, pandas, scikit-learn, FastAPI, SQLite, and Git.

04
Goal of Training
Five key data engineering, ML modeling, and professional competencies.

05
Project Overview
SpendMind vision, problem statement, and proposed automated solution.

06
Working Project
Deep-dive into Extractor, Parser, NLP Classifier, and Dashboard UI.

07
Future Scope
Multi-bank ingestion, active learning, anomaly detection, and cloud scaling.

08
Q & A
Technical defense, architectural inquiry, and faculty evaluation.

ORGANIZATIONAL PROFILE

Company Introduction

Industry host organization supporting the training internship.

ESTABLISHED [YEAR] | HQ: [CITY, STATE, COUNTRY]

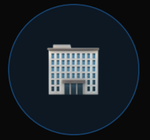
[Organisation Name]

[Organisation Name] is a forward-thinking technology enterprise specializing in [domains / services, e.g. Data Analytics, Cloud Infrastructure, and Financial Systems Engineering].

CORE DOMAINS
[domains / services, e.g. FinTech Data Engineering & AI]

CLIENT BASE
Serving clients across [industries / regions, e.g. BFSI & Tech]

📍 **Office Address:** [Office address, Tech Park / Cyber Hub]



Company Logo

Paste or drag company logo here for formal presentation review

FORMAL VALIDATION

Training Certificate

Official validation of industrial internship completion and practical experience.



Paste Training / Internship Certificate Here

Embed high-resolution scanned copy of the official training completion certificate signed by the mentor and HR authority.

VALIDATION CRITERIA

- ✓ Internship Duration Completed
- ✓ Core Project Milestones Met
- ✓ Code Reviewed & Git Tracked
- ✓ Faculty Assessment Ready

CANDIDATE DETAILS

Somya Roy

Project: SpendMind Data Engine

VERIFIED ITR CANDIDATE

TECHNICAL ECOSYSTEM

Technology Stack Used

Purpose-built, lightweight, and production-grade tools selected for the end-to-end pipeline.

Python 3.11

Core programming language across all pipeline stages: ingestion, NLP vectorization, and REST routing.

pdfplumber

High-precision text and bounding-box table extraction from complex, multi-page bank statement PDFs.

pandas

Tabular data manipulation, transaction reconciliation, schema normalization, and aggregation.

scikit-learn

Machine learning pipeline combining TF-IDF n-gram vectorization with a calibrated Logistic Regression classifier.

FastAPI

Asynchronous REST backend serving upload endpoints, paginated query APIs, and spending summaries.

SQLite + ORM

Zero-config local relational storage managed via SQLAlchemy ORM models for ACID-compliant persistence.

HTML / CSS / JS

Lightweight, zero-dependency analytics dashboard providing rapid category insights and charts.

Git & GitHub

Version control, atomic commit hygiene, branch isolation, and open source repository hosting.

ACADEMIC & INDUSTRY SYNTHESIS

Goal of Industrial Training

Five core engineering competencies targeted and mastered throughout the project lifecycle.

01 **Data Engineering Pipelines** — MASTERED
Build an end-to-end pipeline: raw file ingestion → text extraction → regex parsing → ML categorization → relational storage.

02 **PDF & Unstructured Text Processing** — MASTERED
Handle unstructured bank-statement PDFs using pdfplumber coordinate bounding boxes, text heuristics, and pandas sanitization.

03 **Applied NLP & Machine Learning** — IN PROGRESS
Formulate text tokenization, n-gram TF-IDF vectorization, and train a Logistic Regression classifier for cryptic transaction narrations.

04 **Backend API Development** — PLANNED
Expose processing results through high-performance asynchronous FastAPI REST endpoints backed by SQLite and SQLAlchemy.

05 **Professional Engineering Hygiene** — MASTERED
Master modern software practices: virtual environment isolation, reproducible fixtures, modular code architecture, and Git branching.

PRODUCT ARCHITECTURE

SpendMind Overview

Transforming raw, cryptic financial records into structured intelligence.

"SpendMind turns raw, cryptic bank-statement PDFs into clean, structured, and intelligently categorized financial transaction data."

It extracts text and tabular structures using **pdfplumber**, normalizes each line into Date, Narration, Amount, and Type via **regex heuristics**, and assigns merchants to expense categories using a hybrid **Keyword Rules + TF-IDF ML Classifier**. Results are stored in SQLite and served through FastAPI to an interactive web dashboard.



01 · INPUT

Bank Statement PDF

Raw HDFC-style statements with mixed text and multi-line tables



02 · PROCESS

Extract → Parse → Categorize

pdfplumber layout analysis + heuristic normalizer + NLP ML classifier



03 · OUTPUT

Structured Data & Dashboard

SQLite database, REST API responses, and visual category breakdown charts

MARKET NEED & INNOVATION

Problem Statement & Proposed Solution

Bridging the gap between locked banking documents and modern financial clarity.

The Problem

FRICTION POINT

⚠ **PDF Document Lock-in**
Bank statements arrive as locked, non-standard PDFs that cannot be queried or calculated directly.

⚠ **Manual Spreadsheet Drudgery**
Tracking expenses manually in Excel or Google Sheets is tedious, time-consuming, and highly error-prone.

⚠ **Cryptic Bank Narrations**
Strings like `UPI-ZOMATO-9876543210` or `POS-4482-AMZN` are obscure and inconsistent.

⚠ **Zero Real-Time Cashflow Visibility**
No quick, high-level visualization of where money is actually going without hours of manual sorting.

The SpendMind Solution

ENGINEERED FIX

✓ **Automated Layout Extraction**
Headless extraction of text lines and tabular grids directly from PDF coordinates using pdfplumber.

✓ **Structured Schema Normalization**
Deterministic parsing into clean fields: Date, Clean Narration, Amount, and Type (Dr / Cr).

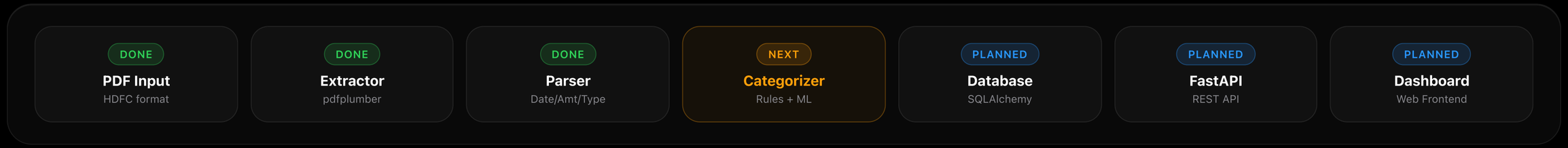
✓ **Hybrid Categorization (Rules → ML)**
Fast keyword heuristics combined with TF-IDF and Logistic Regression to classify novel merchants.

✓ **Queryable Storage & Modern Dashboard**
Zero-latency local SQLite persistence exposed through FastAPI endpoints to an interactive dashboard.

TECHNICAL BLUEPRINT

System Architecture

A modular, loosely-coupled pipeline where each stage is independently verifiable.



Modular

Every stage lives in its own decoupled Python file with explicit inputs and outputs, enabling independent unit testing and debugging.

Reproducible

The synthetic sample statement PDF regenerates byte-for-byte identical via pinned random seeds and deterministic metadata timestamps.

Incremental

Simple deterministic keyword rules provide an immediate baseline; the TF-IDF machine learning classifier is layered on afterward for unseen vendors.

CODEBASE LAYOUT

Working Project: Code Structure

Clean, maintainable directory hierarchy adhering to modern Python packaging standards.

spendmind/ — tree view

```
spendmind/
├── README.md
├── requirements.txt
├── sample_data/
│   └── sample_bank_statement.pdf
└── src/
    ├── api/
    │   └── routes.py
    ├── db/
    │   └── models.py
    ├── ml/
    │   ├── classifier.py
    │   └── rules.py
    ├── pipeline/
    │   ├── extractor.py
    │   └── parser.py
    └── utils/
        └── generate_sample.py
```

src/pipeline/
Dedicated PDF extraction (extractor.py) and multi-column transaction parsing (parser.py).

src/ml/
Two-tier categorization engine: fast keyword lookup tables and scikit-learn TF-IDF classifier.

src/db/
SQLAlchemy database models and SQLite session management for transaction records.

src/api/
FastAPI REST endpoints handling file uploads, query filtering, and category analytics.

src/utils/
Reproducible sample statement PDF generator for testing across heterogeneous merchants.

PIPELINE STAGE 1

Working Project: PDF Extractor Engine

extractor.py — reading raw text streams, bounding coordinates, and table grids from each page.

zsh — python src/pipeline/extractor.py

```
$ python src/pipeline/extractor.py
[INFO] Pages: 3   Lines: 71   Tables: 3
[DATA] Columns: Date | Narration | Amount | Type
[ROW]  03/06/2025 UPI-ZOMATO-9876543210 420.00 Dr
```

3

PAGES INGESTED

Multi-page sequential stream processing

71

TEXT LINES EXTRACTED

Parsed and sanitized line objects

3

TABULAR GRIDS IDENTIFIED

Table bounding boxes segmented

💡

Dates normalized to **DD/MM/YYYY** and Type classified as **Dr / Cr**. Extractor logic was compared across multiple open-source prototypes and consolidated into one robust, tested module.

PIPELINE STAGE 2

Working Project: Transaction Parser

parser.py — converting raw text line streams into structured, reconciled financial records.

65
TRANSACTIONS PARSED
100% parsed with zero drop-off

₹76,193
TOTAL DEBITS VERIFIED
Accurate reconciliation of outflows

₹76,633
TOTAL CREDITS VERIFIED
Accurate reconciliation of inflows

DATA TRANSFORMATION PIPELINE

RAW INPUT LINE

```
03/06/2025 UPI-ZOMATO-9876543210 420.00 Dr
```

→

STRUCTURED JSON OBJECT

```
{"date": "03/06/2025", "narration": "UPI-ZOMATO...", "amount": 420.00, "type": "Dr"}
```

Each parsed record immediately feeds the multi-tier categorization engine in the next stage.

PIPELINE STAGE 3

Working Project: Categorization Engine

A two-tier hybrid architecture combining deterministic rules with natural language processing.

V1.0 · IN PROGRESS

Baseline Engine

Keyword Rules

Matches high-frequency merchant keywords directly from narration tokens:

```
ZOMATO, SWIGGY → Food & Dining
UBER, OLA → Travel & Transit
AMAZON, FLIPKART → Shopping
```

⚡ Fast, deterministic, and requires zero initial training data.

V1.1 · PLANNED

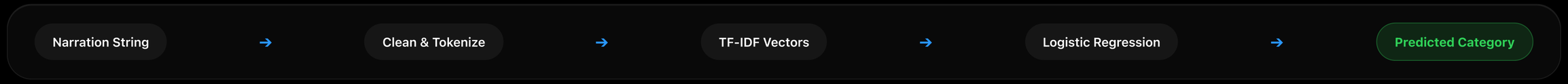
Machine Learning

TF-IDF + Logistic Regression

NLP model capable of generalizing over unseen, noisy bank narration strings:

- TF-IDF converts narration text into n-gram feature vectors
- Logistic Regression predicts the category with confidence scores

🎯 Accurately handles novel merchants that keyword rules have never seen.



END-TO-END SYSTEM

Working Project: Storage, API & Dashboard

Connecting backend data engineering to an intuitive, interactive analytical frontend.

PLANNED

Database

SQLite with SQLAlchemy ORM models.

- Stores date, narration, amount, type & category
- Single local file with zero server setup
- Indexed schema for instant date/category filtering

PLANNED

FastAPI Backend

High-performance async REST microservice.

- `POST /upload` — Process raw PDF statement
- `GET /transactions` — Fetch stored records
- `GET /summary` — Aggregate spend by category

PLANNED

Web Dashboard

Vanilla HTML / CSS / JS visual interface.

- Reuses UI from an earlier Java MVP
- Repointed to high-speed Python backend
- Interactive category pie charts & monthly trends

MILESTONE TRACKER

Development Progress

Current execution status across project phases leading to mid-October 2026 delivery.

01 DONE

Setup & Environment

Virtual env, repo scaffolding, sample PDF generator.

02 DONE

PDF Extractor

pdfplumber line & table extraction verified.

03 DONE

Transaction Parser

65 transactions parsed; debits & credits reconciled.

04 NEXT

ML Categorizer

Heuristic keyword rules + TF-IDF classifier.

05 PLANNED

DB + FastAPI Backend

SQLAlchemy models and async API routes.

06 PLANNED

Analytics Dashboard

Frontend UI connected to live FastAPI backend.

🎯 **Target Delivery:** Mid-October 2026

💻 Code repository & commit logs: **github.com/Missroy1905**

TECHNICAL PROBLEM SOLVING

Engineering Challenges & Solutions

Real-world friction encountered during training and the pragmatic engineering remedies implemented.

PowerShell Execution Policy	Adjusted execution policy scope for process to activate virtual environment without admin overrides.
Unicode Font Encoding in PDF	Resolved ReportLab encoding errors by standardizing Type-1 font mappings for reliable statement generation.
Non-Deterministic PDF Output	Pinned creation date timestamps and metadata headers to produce byte-identical fixtures on every run.
Windows Git Global Config Conflict	Isolated repository credentials using local Git configs (`git config --local`) to ensure clean pushes to GitHub.
Competing Extractor Prototypes	Benchmarked multi-algorithm outputs and merged the most resilient bounding-box extraction logic into one tested module.

VISION & SCALABILITY

Future Scope & Commercial Roadmap

Strategic enhancements to elevate SpendMind from an academic project into an enterprise financial utility.



Multi-Bank Format Support

Expand beyond HDFC to ingest statements from SBI, ICICI, Axis, Kotak, and credit card statement formats.



Adaptive Active Learning

Train the NLP model incrementally as users manually correct or verify merchant category classifications.



Predictive Insights & Alerts

Automated notifications for recurring subscription spikes, unusual transaction amounts, and category budget caps.



Cloud & Multi-User Architecture

Containerization with Docker, PostgreSQL persistence, and secure multi-tenant user authentication.



Tax & ITR Audit Reporting

Automated tax-deductible expenditure tagging (80C, medical insurance, business expenses) with exportable CSV and PDF audit reports directly aligned with Indian Income Tax Return filing standards.

TECHNICAL DEFENSE

Questions & Discussion

Floor open for technical review, architectural defense, and faculty inquiry.

Data Extraction Heuristics

TF-IDF & NLP Classification

FastAPI & SQLite Architecture

CONCLUSION

Thank You

Somya Roy

github.com/Missroy1905

SpendMind — Financial Data Pipeline & NLP-Based Transaction Categorization Engine